Nested Table Example 2: Step-by-Step Explanation with Visuals

```sql
CREATE TYPE proj_t AS OBJECT (
    projno NUMBER,
    projname VARCHAR(15)
);
```

```sql
CREATE TYPE proj_list AS TABLE OF proj_t;
```

```sql
CREATE TYPE employee_t AS OBJECT (
    eno NUMBER,
    projects proj_list
);
```

```sql
CREATE TABLE employees OF employee_t (
    eno PRIMARY KEY
)
NESTED TABLE projects STORE AS employees_proj_table;
```

1. Inserting a Row into the employees Table

```sql
INSERT INTO employees VALUES(1000, proj_list(
    proj_t(101, 'Avionics'),
    proj_t(102, 'Cruise control')
));
```

```sql
SELECT *
FROM TABLE(SELECT t.projects FROM employees t WHERE t.eno = 1000);
```

Use the TABLE(...) function to flatten the nested table and query it like a regular table.

Result:

```
PROJNO   PROJNAME
-------- ---------------
101      Avionics
102      Cruise control
```

1. Unnest or Flatten a Nested Table:
    - Collection unnesting involves "flattening" a nested table by joining each row of the nested table with the row that contains it.
    - This operation is useful for querying individual elements of a nested collection in a relational way.

```
SELECT e.eno, p.*
FROM employees e, TABLE(e.projects) p;
```

`e` : Alias for the `employees` table.

`TABLE(e.projects)` : Flattens the `projects` nested table into a regular table.

`p` : Alias for the flattened table.

`SELECT e.eno, p.*` : Retrieves the employee number (`eno`) and all columns from the flattened `projects` table.

```
ENO     PROJNO    PROJNAME
------  --------  ---------------
1000    101       Avionics
1000    102       Cruise control
2000    100       Autopilot
```

Summary

- **Unnesting:** Convert a nested table into a flat table using the `TABLE(...)` function.
- **Joining:** Combine the flattened nested table with the parent table to retrieve related data.
- **Result:** Retrieve individual elements of the nested collection in a relational format.

DML Operations on Nested Tables

Let's break down how to perform Data Manipulation Language (DML) operations (INSERT, UPDATE, DELETE) on nested tables using the provided examples and clear visuals.

1. **Nested Table Identification:**
    - Use the `TABLE(...)` expression to identify the nested table you want to manipulate.
    - The `TABLE(...)` function converts the nested table into a flat table, allowing you to perform DML operations on it.

1. DML Operations:
    - INSERT: Add new rows to the nested table.
    - UPDATE: Modify existing rows in the nested table.
    - DELETE: Remove rows from the nested table.

2. INSERT Operation

To add a new project (`projno = 103` , `projname = 'Project Neptune'`) to the nested table for `eno = 1000` , use:

```sql
INSERT INTO TABLE(SELECT e.projects FROM employees e WHERE e.eno = 1000)
VALUES (103, 'Project Neptune');
```

- **Explanation:**
  - `SELECT e.projects FROM employees e WHERE e.eno = 1000` : Identifies the nested table for the employee with `eno = 1000` .
  - `TABLE(...)` : Converts the nested table into a flat table.
  - `INSERT INTO ... VALUES (...)` : Adds a new row to the flattened table.

Result:

After the insertion, the `projects` nested table for `eno = 1000` becomes:

```
{proj_t(101, 'Avionics'), proj_t(102, 'Cruise control'), proj_t(103, 'Project Neptu
```

3. UPDATE Operation

To update the `projname` of the project with `projno = 103` to `'Project Pluto'` for `eno = 1000` , use:

```sql
UPDATE TABLE(SELECT e.projects FROM employees e WHERE e.eno = 1000) p
SET p.projname = 'Project Pluto'
WHERE p.projno = 103;
```

- **Explanation:**
  - `SELECT e.projects FROM employees e WHERE e.eno = 1000` : Identifies the nested table for the employee with `eno = 1000` .
  - `TABLE(...)` : Converts the nested table into a flat table.
  - `UPDATE ... SET ... WHERE ...` : Updates the `projname` of the row where `projno = 103` .

Result:

After the update, the `projects` nested table for `eno = 1000` becomes:

```
{proj_t(101, 'Avionics'), proj_t(102, 'Cruise control'), proj_t(103, 'Project Pluto
```

4. DELETE Operation

To remove the project with `projno = 103` from the nested table for `eno = 1000`, use:

```sql
DELETE FROM TABLE(SELECT e.projects FROM employees e WHERE e.eno = 1000) p
WHERE p.projno = 103;
```

- **Explanation:**
  - `SELECT e.projects FROM employees e WHERE e.eno = 1000` : Identifies the nested table for the employee with `eno = 1000` .
  - `TABLE(...)` : Converts the nested table into a flat table.
  - `DELETE ... WHERE ...` : Deletes the row where `projno = 103` .

Result:

After the deletion, the `projects` nested table for `eno = 1000` becomes:

```
{proj_t(101, 'Avionics'), proj_t(102, 'Cruise control')}
```

- **Dropping a Nested Table: Set the nested table column to NULL using an UPDATE statement.**

1. Initial Setup

Assume the `employees` table has the following structure:

ENO	PROJECTS
1000	{proj_t(101, 'Avionics'), proj_t(102, 'Cruise control')}
2000	{proj_t(100, 'Autopilot')}

The `projects` column is a nested table of type `proj_list` .

After Dropping the Nested Table

```sql
UPDATE employees e
SET e.projects = NULL
WHERE e.eno = 1000;
```

Result:

ENO	PROJECTS
1000	NULL
2000	{proj_t(100, 'Autopilo

- **Adding Back a Nested Table Row: Update the nested table column with a new value using the proj_list constructor.**

After Adding Back a Nested Table Row

```sql
UPDATE employees e
SET e.projects = proj_list(proj_t(103, 'Project Pluto'))
WHERE e.eno = 1000;
```

Result:

ENO	PROJECTS
1000	{proj_t(103, 'Project Pluto')}
2000	{proj_t(100, 'Autopilot')}

Key Points About DML on Nested Tuples

1. **Difference Between `NULL` and Empty Constructor:**
   - `NULL`: When you set a nested table column to `NULL`, it completely removes the nested table from the parent row.
   - **Empty Constructor (`proj_list()`):** When you set a nested table column to an empty constructor, it creates an empty nested table (a nested table with zero rows) but retains the structure.
2. **Adding Back a Nested Table Row:**
   - You can add back a nested table row in two steps:
     1. First, create an empty nested table using the constructor (`proj_list()`).
     2. Then, insert rows into the nested table using the `INSERT INTO TABLE(...)` syntax.

↓

sql

```sql
UPDATE employees e
SET e.projects = proj_list()
WHERE e.eno = 1000;
```

sql

```sql
INSERT INTO TABLE(SELECT e.projects FROM employees e WHERE e.eno = 1000)
VALUES (proj_t(102, 'Project Pluto'));
```

- **Multilevel Collection Types:** Collections whose elements are themselves another collection type.
- **Types of Multilevel Collections:**
  - Nested Table of Nested Table Type
  - Nested Table of VARRAY Type
  - VARRAY of Nested Table Type
  - VARRAY of VARRAY Type
  - Nested Table or VARRAY of a UDT with a nested table or VARRAY attribute.
- **Use Case:** Modeling hierarchical or multidimensional data structures.

Multilevel Collection Types Example

1. Problem Statement

The goal is to model a system where:

- Each **star** has a collection of **planets** revolving around it.
- Each **planet** has a collection of **satellites** orbiting it.

This requires a **multilevel collection structure** because:

- The `satellites` attribute of a planet is a collection (nested table) of satellites.
- The `planets` attribute of a star would be a collection (nested table or VARRAY) of planets.

(a) Define the Satellite Type (`sat_t`)

First, define an object type called `sat_t` to represent a satellite. A satellite has:

- `name` : The name of the satellite (e.g., "Moon").
- `orbit` : The orbital distance from the planet (in arbitrary units).

sql

```sql
CREATE TYPE sat_t AS OBJECT (
    name VARCHAR2(20),
    orbit NUMBER
);
/
```

- **Explanation:**
  - `sat_t` : The name of the object typ

(b) Define the Satellite Nested Table Type (`sat_ntt`)

Next, create a nested table type called `sat_ntt` that represents a collection of satellites.

sql

```sql
CREATE TYPE sat_ntt AS TABLE OF sat_t;
/
```

- **Explanation:**
  - `sat_ntt` : The name of the nested table type.
  - `TABLE OF sat_t` : Specifies that this nested table can store multiple instances of the `sat_t` object type.

(c) Define the Planet Type (`planet_t`)

Now, define an object type called `planet_t` to represent a planet. A planet has:

- `name` : The name of the planet (e.g., "Earth").
- `mass` : The mass of the planet (in arbitrary units).
- `satellites` : A nested table of satellites orbiting the planet.

```sql
CREATE TYPE planet_t AS OBJECT (
    name VARCHAR2(20),
    mass NUMBER,
    satellites sat_ntt
);
/
```

- **Explanation:**
  - `planet_t` : The name of the object type

3. Multilevel Collection Structure

The example demonstrates a **two-level hierarchy**:

1. **Satellite Level:** Each satellite is represented by the `sat_t` object type.
2. **Planet Level:** Each planet is represented by the `planet_t` object type, which includes a nested table of satellites (`sat_ntt`).

This structure allows you to model:

- Multiple satellites for each planet.
- Multiple planets for each star (though the star type is not defined in this example).

4. Extending the Example

If you wanted to extend this to include stars, you could define a `star_t` object type with a nested table of planets:

```sql
CREATE TYPE star_t AS OBJECT (
    name VARCHAR2(20),
    age NUMBER,
    planets planet_ntt
);
/
```

Where `planet_ntt` is a nested table of `planet_t` objects.

5. Summary

- **Satellite (`sat_t`):** Represents a single satellite with a name and orbital distance.
- **Satellite Nested Table (`sat_ntt`):** Represents a collection of satellites.
- **Planet (`planet_t`):** Represents a single planet with a name, mass, and a nested table of satellites.

This example illustrates how multilevel collection types can be used to model hierarchical data structures efficiently.